The Token Tax of the Web

Vedang Ratan Vatsa
veda.ng

Abstract

Sending a webpage's HTML to a language model uses tokens for tags as well as the page's readable text. Removing tags can shorten the input, but can also remove table structure needed to interpret the text. This study tests whether shorter webpage formats save tokens without losing information needed to answer questions. In the saved-page test, Markdown used 49.6% fewer recorded input tokens than cleaned HTML and matched the same number of expected answers. A separate review checked differing replies against the original pages for errors and missing information.

Keywords: webpage formats, tokens, HTML, Markdown, question answering

1. The Problem

HTML tags identify headings, links, and table cells. For example, a price can appear as `<td>16.99</td>`, where `<td>` and `</td>` mark a table cell. A language model given this HTML receives both the price and the tags.

A **language model** takes text and writes a reply. **Input** is the text sent to it; **output** is its reply. The model counts text in **tokens**. A token can be a word, part of a word, or punctuation. The HTML tags count toward the input too [6]. The title's "token tax" refers to the tokens used for this markup.

The test used five forms of the same webpage parts. Markdown used 49.6% less recorded request input than cleaned HTML, and both matched the same number of stored answers. Each format received the same questions and GPT settings. The input counts include the page text and the full request. Differing replies were checked against the saved pages.

2. The Test

Material

WebSRC is a collection published in 2021. It contains saved webpage parts, questions, and expected answers [1]. Its creators had already selected page areas and removed some code. The test chose six parts from each of 50 collection groups, covering 43 website addresses.

Topic	Page parts
Cars	30
Books	30
Cameras	12
Games	12
Jobs	30
Films	18
Phones	24
Restaurants	18
Sports	72
Universities	54
Total	**300**

Each part had up to three questions. There were 291 parts with three questions, six with two, and three with one, making 888 questions. Rewordings about the same stored answer and location were grouped before selection. The questions and expected answers came from WebSRC.

Formats

Each part appeared in five forms.

1. **Saved HTML.** The text with the page marks retained in WebSRC.
2. **Cleaned HTML.** The same part with more code and page labels removed, while keeping its basic arrangement.
3. **Markdown.** Text that uses short marks for headings, lists, links, and simple tables.
4. **Plain text.** Words and numbers in their original order, with most page marks removed.
5. **Nested JSON.** A labelled list of page parts and the smaller parts inside them. JSON is a format for storing structured information.

The conversion used fixed rules rather than AI rewriting. Markdownify made the Markdown version [7]. The rules did not use the questions or expected answers to choose what to keep. The collection's research-only answer-location labels were removed first.

Procedure

OpenCode sent the requests to the existing model named `gpt-6.1-sol` [8]. The main run took place on 12 October 2026 UTC. Each version used the same questions and requested settings.

The model received one page part and its questions in a fresh chat. It was asked to copy the shortest passage that answered each question, keep units, and leave the answer empty when the text did not support one. Questions came before the page text. Browsing and file access were disabled.

The 300 parts in five formats made 1,500 main requests. Asking all 888 questions in each format produced 4,440 answer attempts. The test also included a preliminary run and a repeat run.

Run	Page parts	Requests
Preliminary check	30 different parts	150
Main test	300 parts	1,500
Repeat check	50 parts from the main test	250
Total test requests		**1,900**

The result tables use the main run. Before collecting its replies, a written plan chose two main measures. One was the middle saving across the 300 parts. The other was the average answer score for those parts.

Marking and counting

An **answer-key match** means that the reply agrees with the stored expected answer after text cleanup. The program treated page spellings such as `&` as the symbol `&`. It also put equivalent character forms into a common form and ignored letter case and extra spaces. Numbers, signs, units, and punctuation otherwise remained. For example, 1.5 did not match 15.

The study counted two kinds of input.

- **Page text** counts the tokens in the chosen page format, using one fixed public counting rule [6].
- **Complete input** is the count recorded for the full request. It includes the page, questions, instructions, and other request text.

These input counts exclude the reply. Exact settings and conversion details appear in Section 7.

3. Results

Input and answers

Each row covers the same 300 parts and 888 questions.

Format	Page-text tokens	Complete input tokens	Key matches, out of 888
Saved HTML	625,455	724,469	805
Cleaned HTML	286,312	385,326	798
Markdown	94,874	194,035	798
Plain text	67,658	166,936	784
Nested JSON	348,183	447,197	795

Markdown matched the same 798 answers as cleaned HTML with less input. Plain text used still less input but matched 14 fewer answers than Markdown. Saved HTML matched seven more answers and used more than three times as much complete input. Figure 1 compares page and request counts for Markdown and cleaned HTML.

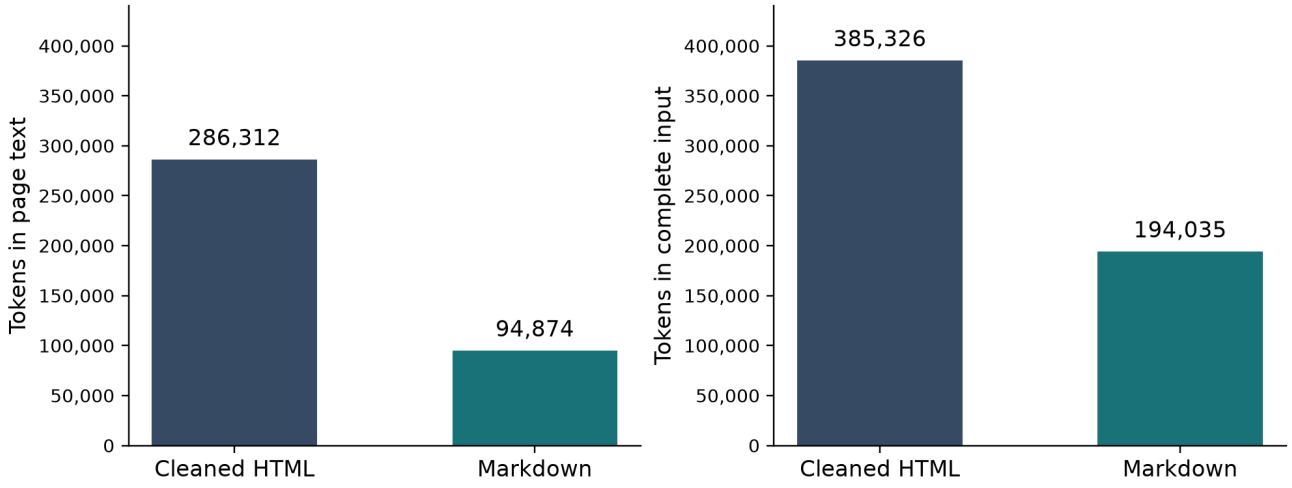


Figure 1: Token counts for the same 300 parts. The left panel counts page text. The right panel includes questions and instructions. Markdown reduced the page-text total by 66.9% and complete input by 49.6%.

Savings

Page text fell from 286,312 to 94,874 tokens. The difference is 191,438. Dividing that difference by the starting count gives a 66.9% saving.

Complete input fell from 385,326 to 194,035 tokens. The difference is 191,291, or 49.6%. Questions and instructions remain in both versions, so the page change removes a smaller share of the full request.

For the page-by-page measure, the study found the percentage saving for each part and sorted the 300 results. The **median** is the average of the two middle percentages. It was 68.6%. The combined 66.9% saving gives longer parts more influence; the median describes the middle of the page-by-page savings.

The main answer score gave each part equal weight. Each part's score was its matches divided by its questions. Averaging the 300 scores gave 90.00% for both formats. Counting every question equally instead gives 798 divided by 888, or 89.86%.

Differing replies

Result on a question	Questions
Both formats matched the key	789
Only Markdown matched	9
Only cleaned HTML matched	9
Neither matched	81
Total	**888**

Each format's total is 789 plus nine, or 798. The stored replies differed on 45 questions. On 23 of these, the difference disappeared under the normal text cleanup. The remaining 22 differed after cleanup as well.

4. Answer Review

Review rule

The paper's AI assistant inspected all 45 differing pairs against the original saved text and table labels. It also checked five screenshots where layout or labels affected the judgment. The format names and published keys were visible during review.

The review used five labels.

- **Correct and complete.** The source supports the reply, and it answers the question. Extra true detail is allowed.
- **Incomplete.** The reply gives useful information but leaves out a needed condition or detail, or leaves competing values unresolved.
- **Wrong.** The reply contradicts the source or selects the wrong fact.
- **No answer.** The reply is empty.
- **Unclear.** The question or source leaves the intended answer uncertain.

The other 843 pairs had identical stored replies and were outside this meaning review.

Judgments

Review label	Cleaned HTML	Markdown
Correct and complete	39	40
Incomplete	1	1
Wrong	0	0
No answer	3	2
Unclear	2	2
Reviewed replies	**45**	**45**

Markdown had one more correct and complete reply in the reviewed set. Some replies gave the right fact in different words. Others left out a condition or gave more than one price.

Warranty

Expected answer

> 36 mo. / 36,000 mi. Warranty

Cleaned-HTML reply

> 36 mo. / 36,000 mi.

Both replies give the same 36-month and 36,000-mile limits. The review accepted both. The strict key check rejected the shorter reply because it omitted "Warranty."

Salary and price

The salary depended on experience.

> 41,526 to 51,034 per annum depending on relevant experience

Per annum means per year.

Cleaned HTML gave the annual range but left out the experience condition. It was rated incomplete. Markdown kept the condition and was rated correct and complete.

For a book price, cleaned HTML gave 16.99. Markdown returned two amounts.

> 20.00 16.99

The screenshot crosses out 20.00 and shows 16.99 as the current price. The review accepted the cleaned-HTML reply. It rated Markdown incomplete because the reply did not choose between the two amounts.

Short and uncertain answers

A camera answer gave “660 g” while the key also listed pounds and ounces. The review accepted the gram value as a complete answer to the weight question. It also accepted longer car names that included true trim details.

Two roster questions asked for birthplace, but their source columns said hometown. Those location replies were marked unclear. Another question asked for a device’s “optical resolution.” The page showed both screen and camera resolution, leaving the intended field uncertain.

5. Extra Checks

Repeats

The repeat run used one selected part from each collection group, giving 50 parts and 149 questions. Cleaned HTML matched 132 answers in each run; Markdown matched 130. Three cleaned-HTML replies and two Markdown replies changed between runs while those totals stayed the same.

Website mix

The analysis repeated the calculation 10,000 times using different mixes of the 43 observed website addresses. It drew addresses with repeats and kept each site’s parts together. These were new calculations using the saved replies.

The middle 95% of the differences ran from 0.87 percentage points lower to 0.87 points higher for Markdown. A percentage point is one point on a percentage scale; moving from 90% to 91% adds one point. The input savings in these recalculations ranged from about 45.1% to 53.1% across the middle 95%.

Two addresses appeared in both the preliminary and main sets under different group labels. Removing their main parts left 288 parts at 41 addresses. The average section scores still tied.

Cleaning and counting

Further HTML cleaning reduced the page count from 286,312 to 269,960 tokens. Markdown was still 64.9% shorter in total. The extra cleaning check counted text only.

A second public token-counting rule gave a 68.4% median page saving and a 66.7% combined saving. The original rule gave 68.6% and 66.9%.

Scoring also depended on text cleanup. Cleaning only letter case and spaces gave 791 key matches for cleaned HTML and 798 for Markdown. Decoding HTML character spellings, such as `&` to `&`, changed 15 of the 4,440 marks across the five formats. The main tables use the original full cleanup rule.

6. Contribution

HtmlRAG chooses and shortens page content [2]. ReaderLM-v2 trains a model to convert content [3]. Other studies test webpage descriptions used by agents [4] and table formats [5]. This test used fixed rules to convert given page parts.

The same questions were asked for each page format. With the same model name and marking rule, Markdown matched the cleaned-HTML answer-key total while using 49.6% less complete input. The source review compared differing replies with the original pages. It recorded omitted conditions, competing prices, empty replies, and questions that the page could not settle.

The page text fell by 66.9%, but the full request fell by 49.6% because questions and instructions remained. Each request supplied one page part once for all its questions.

7. Study Details

The group names come from WebSRC.

Item	Setting or rule
Source groups	Six parts from each of 50 WebSRC training groups; three parts from each of ten development groups for the preliminary run
Selection	Random starting number 20261013; the expected text had to appear at its recorded location, or be a stored yes/no answer
Text size	Every chosen version fit within 64,000 tokens; the selected text was kept in full
Reader	`gpt-6.1-sol` through OpenCode 1.17.11; requested reasoning and reply detail both low
Run date	12 October 2026 UTC
Order	Fixed shuffle starting number 20261015; at most four requests at once
Instructions	Copy the shortest passage answering the question, keep units, and use an empty answer when needed
Main counts	Public `o200k_base` rule in tiktoken 0.14.0 for page text; OpenCode's recorded count for complete input
Second counter	Public `cl100k_base` rule
Conversion tools	Beautiful Soup 4.15.0 and Markdownify 1.2.3
Text cleanup for marking	Decode HTML character spellings; standardize equivalent character forms using NFKC; ignore letter case and repeated spaces
Planning	Sample size, main comparison, marking rule, and main measures were written down after the preliminary run and before the main replies; the plan was local and unregistered
Later checks	The source review and extra-cleaning check used the saved material after the main results were known

Cleaned HTML removed comments, code used to run or style the page, and labels used to identify page parts. It also removed fallback text meant to appear when scripts are off. It kept the remaining arrangement and labels for links, tables, images, titles, dates, language, and roles. Plain text kept the words in order. Nested JSON kept a labelled list of the remaining parts. The converter chose content without seeing the questions or key.

Limits

The result concerns prepared page parts from an older collection and one GPT setup. The model behind its name may change, and it may have seen WebSRC during earlier training. The public counting rules are fixed text measures; the model's internal rule was unknown.

Shared errors can remain among the identical replies. Website recalculations describe changes in this sample's mix, while the repeat run covers only its selected parts.

The study measured input text and answer agreement. Bills, energy use, and the speed effect of shorter input require separate measurements. The recorded middle reply times were about 2.7 to 2.8 seconds and included network and service delays.

8. Conclusion

Markdown matched the same 798 stored answers as cleaned HTML while using 49.6% less complete input. Its combined page count was 66.9% lower, and its middle page-by-page saving was 68.6%. In the 45 differing pairs, source review judged 39 cleaned-HTML replies and 40 Markdown replies correct and complete.

Appendix A. Language Counts

A separate check counted published human translations from FLORES-200 [9]. It used all 1,012 sentences in the original public `devtest` set, drawn from 281 English-language articles. The set contains 341 Wikinews sentences, 351 Wikibooks sentences, and 320 Wikivoyage sentences. The selection covers 12 languages, with two Chinese scripts, making 13 versions.

Rule A is the public counting rule used for the page results, `o200k_base`. **Rule B** is `cl100k_base`. Both belong to the tiktoken tool [6]. A rule decides which pieces of text count as one token. The table compares each version's total with English. A plus sign means more tokens.

Language version	Extra tokens, Rule A	Extra tokens, Rule B
English	Reference	Reference
Chinese (simplified)	+24.9%	+86.9%
Chinese (traditional)	+41.8%	+114.1%
Spanish	+31.8%	+54.4%
French	+37.0%	+60.8%
German	+31.1%	+58.6%
Portuguese	+22.5%	+47.8%
Indonesian	+24.9%	+55.1%
Arabic	+38.1%	+202.7%
Hindi	+57.4%	+375.9%
Japanese	+65.8%	+126.6%
Korean	+47.2%	+136.7%
Russian	+42.1%	+146.2%

English used 26,873 tokens under Rule A. Simplified Chinese used 33,555, or 24.9% more. Under Rule B, the counts were 27,182 and 50,811, a 86.9% increase. Chinese had fewer written characters but more tokens in both totals.

With Rule A, 116 individual Chinese sentences used fewer tokens than their English versions, even though the combined Chinese count was higher. Joining all sentences into one text for each language

also left every tested translation with more tokens than English.

The program counted the published translations as written. The source texts were in English. The Chinese versions can use different words as well as different scripts. This table measures text size; it contains no model-answer or translation-cost test.

References

1. Chen, Xingyu, Zihan Zhao, Lu Chen, JiaBao Ji, Danyang Zhang, Ao Luo, Yuxuan Xiong, and Kai Yu. "WebSRC: A Dataset for Web-Based Structural Reading Comprehension." *Proceedings of EMNLP*, 2021, pp. 4173-4185. aclanthology.org/2021.emnlp-main.343. Dataset release, huggingface.co/datasets/X-LANCE/WebSRC_v1.0 (accessed October 2026).
2. Tan, Jiejun, Zhicheng Dou, Wen Wang, Mang Wang, Weipeng Chen, and Ji-Rong Wen. "HtmlRAG: HTML is Better Than Plain Text for Modeling Retrieved Knowledge in RAG Systems." *The ACM Web Conference*, 2025. arxiv.org/abs/2411.02959.
3. Wang, Feng, Zesheng Shi, Bo Wang, Nan Wang, and Han Xiao. "ReaderLM-v2: Small Language Model for HTML to Markdown and JSON." arXiv preprint, 2025. arxiv.org/abs/2503.01151.
4. Enomoto, Masafumi, Ryoma Obara, Haochen Zhang, and Masafumi Oyamada. "Read More, Think More: Revisiting Observation Reduction for Web Agents." arXiv preprint, 2026. arxiv.org/abs/2604.01535.
5. Ahsan, Momina, Sarfraz Ahmad, Ming Shan Hee, Roy Ka-Wei Lee, and Preslav Nakov. "TABVERSE: Benchmarking Cross-Format Table Understanding in LLMs and VLMs." arXiv preprint, 2026. arxiv.org/abs/2606.09578.
6. OpenAI. "tiktoken." Software and documentation. github.com/openai/tiktoken (accessed October 2026).
7. python-markdownify contributors. "python-markdownify." Software and documentation. github.com/matthewwithanm/python-markdownify (accessed October 2026).
8. OpenCode. "Server." Software documentation. opencode.ai/docs/server (accessed October 2026).
9. NLLB Team. "No Language Left Behind: Scaling Human-Centered Machine Translation." arXiv preprint, 2022, Section 4.1. arxiv.org/abs/2207.04672. Original FLORES-200 dataset documentation, github.com/facebookresearch/flores/tree/main/flores200 (accessed October 2026).